



Repositorios de Software

UNIDAD 1
METODOLOGÍA DE SISTEMAS II

Contenido

Introducción.....	3
Concepto de Repositorio	3
Concepto de Repositorio de Software.....	3
Características de los repositorios	4
Almacenamiento.....	4
Control de versiones.....	5
Colaboración	5
Organización.....	5
Control de acceso.....	5
Integración	5
Trazabilidad.....	5
Gestión de ramas	5
Revisión de código	6
Automatización.....	6
Gestión de incidencias	6
Documentación.....	6
Tipos de Repositorios.....	6
Repositorio del Sistema de Control de Versiones	6
Repositorios del gestor de paquetes	7
Repositorios de datos	7
Repositorios de infraestructura como código (IaC)	7
Repositorios de artefactos	8
Evolución de los Sistemas de Control de Versiones	8
Repositorios Centralizados	9
Repositorio Distribuido	10
Repositorio centralizado vs. repositorio distribuido	11
Características de un repositorio	11
Control de versiones:.....	11
Historial de cambios (log):.....	11
Trazabilidad:.....	11
Ramificación (branching):	11
Fusión (merging):	11
Reversión (rollback):.....	11
Colaboración:	11
Backup/respaldo:.....	11

Gestión de conflictos:	11
Clasificación de repositorios	12
Según la arquitectura:	12
Según el acceso:	12
Según el alojamiento:	12
Según el contenido/propósito:	12
Plataformas de repositorios	12
GitHub	12
GitLab	13
Bitbucket	14
Buenas prácticas para la utilización de repositorios	15
Importancia de los repositorios en la Ingeniería de Software	17
Conclusión	18
Bibliografía:	18

UNIDAD 1 -Herramientas de repositorios de software

Contenidos:

Conceptos de repositorio. Repositorio centralizado y repositorio distribuido. Características y Funciones de un repositorio. Clasificación de repositorios. Plataforma: GitHub, GitLab, Bitbucket.

Introducción

El desarrollo de software constituye una actividad esencialmente colaborativa. Los sistemas actuales suelen ser construidos por equipos integrados por desarrolladores, analistas, testers, responsables de infraestructura, administradores de proyectos y otros profesionales que trabajan simultáneamente sobre diferentes componentes de una solución tecnológica.

En este contexto, uno de los principales desafíos consiste en administrar adecuadamente los cambios realizados sobre el código fuente y sobre los diferentes artefactos que forman parte de un proyecto. Cuando varias personas modifican los mismos archivos sin mecanismos adecuados de control, pueden producirse pérdidas de información, sobrescritura de cambios, inconsistencias y dificultades para determinar quién realizó una modificación, cuándo la realizó y cuál fue su propósito.

Los sistemas de control de versiones surgieron precisamente para resolver estos problemas. Su función fundamental es registrar los cambios realizados sobre archivos a lo largo del tiempo, permitiendo recuperar versiones anteriores, comparar modificaciones y conocer la evolución de un proyecto.

Dentro de este ámbito adquiere especial importancia el concepto de **repositorio de software**, entendido como el espacio destinado a almacenar y administrar los archivos de un proyecto junto con la información relacionada con su evolución.

La evolución de los sistemas de control de versiones dio lugar a diferentes modelos de organización. Inicialmente aparecieron soluciones locales; posteriormente se desarrollaron sistemas centralizados, donde un servidor concentra la información; y finalmente surgieron los sistemas distribuidos, como Git, en los cuales cada copia clonada del repositorio contiene la historia completa del proyecto.

Concepto de Repositorio

Un repositorio es un espacio centralizado o digital donde se almacenan, organizan, mantienen y comparten archivos o datos. Dependiendo del contexto, puede ser un lugar para guardar código fuente de programas (como en GitHub), documentos académicos o investigaciones científicas

Ejemplo: RIA-RIUNNE

Concepto de Repositorio de Software

Un repositorio de software es un espacio de almacenamiento donde se guardan y organizan los archivos de un proyecto (código fuente, documentación, recursos, etc.), junto con el historial completo de cambios realizados sobre ellos.

No es solo un "lugar donde guardar archivos": lleva un registro de quién cambió qué, cuándo y por qué (a través de los mensajes de commit).

Un repositorio puede contener:

- Código fuente.
- Archivos de configuración.
- Documentación.
- Scripts.
- Pruebas automatizadas.
- Archivos de instalación.
- Archivos de configuración de CI/CD.
- Diagramas y documentación técnica.
- Archivos de licencia.
- Archivos README.
- Recursos necesarios para construir o ejecutar el sistema.

Por lo tanto, el repositorio constituye uno de los elementos centrales de la gestión técnica de un proyecto de software.

Características de los repositorios



Almacenamiento

Un repositorio guarda todos los archivos de tu proyecto en un solo lugar. En lugar de tener el código en un ordenador portátil, las imágenes en un hilo de chat y las notas en un correo electrónico, todo se encuentra en un solo lugar.

Ejemplo: Si se está creando un sitio web, por ejemplo, tu HTML, CSS, imágenes y notas se almacenan en el mismo repositorio. De esta manera, no hay confusión ni archivos perdidos.

Control de versiones

Cada vez que se guardan cambios en un repositorio, este los recuerda. Es posible ver quién realizó la edición, cuándo la hizo y qué se modificó. Si ocurre algo inusual, siempre puedes volver a una versión anterior.

Ejemplo: Suponga que su compañero de equipo borra accidentalmente parte del código de la página de inicio. (Lo sé, ¿verdad?) En lugar de reescribir todo, se puede regresar a la versión de ayer y el problema se resuelve.

Colaboración

Si se trabaja en un repositorio, el equipo puede contribuir al mismo tiempo, es decir que se puede trabajar simultáneamente. No tienes que esperar a que otra persona termine, ni preocuparse por sobrescribir el trabajo de los demás. Todos pueden centrarse en su trabajo, y el repositorio lo mantiene todo sincronizado.

Organización

Con un repositorio, los archivos no estarán dispersos ni serán difíciles de encontrar. Se puede configurar una estructura de carpetas clara para saber siempre exactamente dónde están los archivos y mantener el trabajo ordenado evitando perder tiempo buscando ese archivo que se requiere.

Control de acceso

A veces se quiere mantener el proyecto en privado y otras veces se requiere compartirlo. Un repositorio permite controlar quién puede ver o editar un trabajo.

Por ejemplo, se puede mantener un repositorio privado mientras se sigue desarrollando y luego hacerlo público después del lanzamiento para que otros puedan explorar el proyecto terminado.

Integración

Los repositorios también pueden conectarse con otras herramientas que se utiliza. Esto significa que se puede configurar por ejemplo flujos de trabajo que prueben el código automáticamente, implementen actualizaciones o incluso actualicen tus tareas de gestión de proyectos sin que tengas que mover un dedo.

Suponga que se envía código nuevo al repositorio. De inmediato, se ejecutan pruebas automatizadas y se señalan los errores. Al mismo tiempo, el tablero de proyectos marca la tarea como completada. De esa manera, todo funciona en conjunto.

Trazabilidad

Permite identificar qué cambios fueron realizados, cuándo y, dependiendo de la configuración y del flujo de trabajo, por quién.

Gestión de ramas

Las ramas permiten desarrollar funcionalidades o correcciones de manera independiente sin modificar inmediatamente la línea principal. Una rama puede representar una versión paralela del código.

Revisión de código

Las modificaciones pueden someterse a procesos de revisión antes de incorporarse a la rama principal.

Automatización

Los repositorios pueden integrarse con herramientas de integración continua y entrega continua.

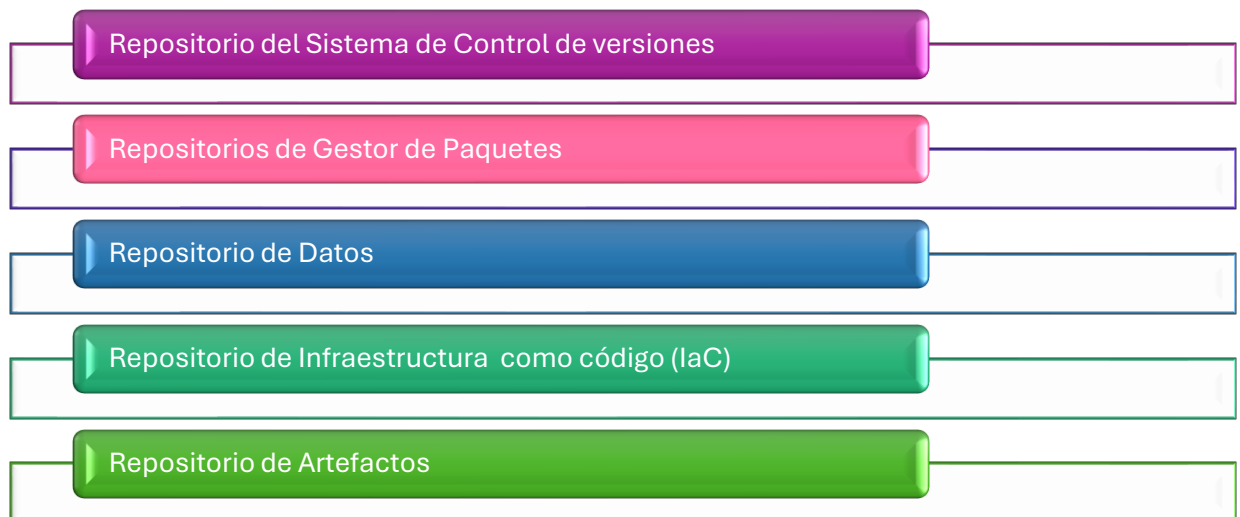
Gestión de incidencias

Las plataformas pueden incorporar herramientas para registrar errores, solicitudes y tareas.

Documentación

Los repositorios permiten incorporar archivos como README, licencias, guías de contribución y documentación técnica.

Tipos de Repositorios



Repositorio del Sistema de Control de Versiones

Los VCS (version control system) son aplicaciones que permiten realizar el control de modificaciones de código fuente de manera automática y eficiente.

En el contexto de los sistemas de control de versiones, el repositorio no debe entenderse simplemente como una carpeta donde se guardan archivos, ya que permite:

- Rastrear la evolución de un proyecto a lo largo del tiempo.
- Identificar los cambios realizados
- Volver a versiones anteriores si algo sale mal.
- Trabajar en equipo sin sobrescribir el trabajo de otros.
- Comparar cambios entre distintas versiones.
- Reporte de cambios: Cuando un archivo es modificado se guarda la fecha y hora del cambio, el autor que lo realizó y un mensaje opcional explicando las razones o naturaleza del cambio.
- Sincronización: Si varias personas trabajan o utilizan el mismo proyecto permite que todas puedan tener la última versión

-
- Backup y restauración: Permite guardar los cambios realizados en los archivos y restaurar el archivo a cualquiera de los estados previos en los que fue guardado
 - Crear branch: Permite crear una branch (rama) del proyecto y trabajarla por separado sin modificar el proyecto original.
 - Realizar merge: Permite convertir el contenido de branch al proyecto principal

Repositorios del gestor de paquetes

Un repositorio de gestor de paquetes almacena bibliotecas de código (denominadas *paquetes*). En lugar de escribir todo desde cero, se puede utilizar código ya creado para acelerar el desarrollo. También se puede empaquetar el proyecto propio y compartirlo con otros.

Esto ahorra tiempo y mantiene la coherencia, ya que no se debe programar nuevamente funciones comunes como sistemas de inicio de sesión, gráficos o botones.

A continuación, se muestran algunos repositorios populares de gestores de paquetes:

- [npm](#) es el principal gestor de paquetes para JavaScript. Aloja miles de bibliotecas como React, Angular y Bootstrap.
- [PyPI \(Python Package Index\)](#) es el centro neurálgico de los paquetes Python. Puedes instalar estas bibliotecas utilizando pip.
- [apt](#) se utiliza habitualmente en sistemas Linux para instalar y actualizar software a nivel del sistema operativo.

Repositorios de datos

Un repositorio de datos es un lugar centralizado donde se almacenan, gestionan y comparten conjuntos de datos. En lugar de mantener archivos enormes dispersos en diferentes computadoras, un repositorio de datos los guarda en una ubicación segura.

Una de sus principales ventajas es que los repositorios de datos suelen incluir *metadatos* (detalles como el autor, la fecha, la versión y la licencia). Esto hace que los datos sean más fáciles de entender, reutilizar y citar.

Un ejemplo muy conocido de repositorio de datos es [Kaggle](#), donde las personas suben conjuntos de datos para que otras personas los descarguen, analicen e incluso mejoren de alguna manera.

Repositorios de infraestructura como código (IaC)

Un repositorio de infraestructura como código(IaC) es donde se almacenan los archivos de configuración que describen la infraestructura, como servidores, redes o recursos en la nube. En lugar de configurarlos manualmente, los escribes como código.

Porque una vez que la infraestructura es código, se puede versionar, revisar, reutilizar y automatizar como si fuera software. Esto hace que los equipos de DevOps sean más rápidos y menos propensos a cometer errores humanos.

Por ejemplo, con [Terraform](#), se puede escribir archivos de configuración sencillos que indican qué infraestructura se necesita para el proyecto. A continuación, Terraform lo compila o

actualiza automáticamente. Esa configuración permanece en el repositorio para que todo el equipo pueda verla, programar los cambios y reutilizarla en proyectos futuros.

Repositorios de artefactos

Estas herramientas están diseñadas para almacenar y gestionar los artefactos binarios (los resultados compilados del código, como archivos .jar, paquetes .npm, imágenes .docker, etc.). Son cruciales para los *pipelines* de CI/CD. Cuando el código se compila o empaqueta (por ejemplo, en .jar, .exe, .npm, .docker), se necesita un repositorio para almacenar estos **artefactos binarios**.

Su función principal es garantizar la inmutabilidad de los artefactos y actuar como un punto central de verdad para todas las dependencias y entregables.

Ejemplos:

- **Nexus Repository Manager**
- **JFrog Artifactory**
- **GitHub Packages**
- **Docker Hub**

Estos repositorios se integran con los de código para gestionar todo el ciclo de vida del software.

Herramienta	Función Principal	Tipos de Artefactos Comunes
JFrog Artifactory	Gestión universal de binarios, compatibilidad con más de 30 tipos de paquetes (Maven, npm, Docker, PyPI, NuGet, etc.).	Paquetes de código abierto y privados, imágenes Docker.
Sonatype Nexus Repository	Similar a Artifactory, permite alojar, almacenar en caché (proxy) y gestionar dependencias de proyectos (especialmente Maven/Java).	Paquetes Maven, npm, Docker, NuGet.
AWS CodeArtifact	Servicio de gestión de artefactos completamente gestionado por AWS, compatible con Maven, npm, PyPI y NuGet.	Paquetes de software para entornos AWS.
Container Registries (como Docker Hub, Google Container Registry, Azure Container Registry)	Especializados en el almacenamiento y distribución de Imágenes de Contenedores Docker .	Imágenes Docker, Gráficos Helm.

Evolución de los Sistemas de Control de Versiones

La necesidad de conservar diferentes versiones de un proyecto surgió prácticamente con el desarrollo de software colaborativo.

Una **estrategia inicial** consistía simplemente en realizar copias de los archivos y almacenarlas en diferentes carpetas. Por ejemplo:

- Proyecto_v1
- Proyecto_v2
- Proyecto_v3
- Proyecto_final
- Proyecto_final_definitivo

Aunque este mecanismo puede parecer sencillo, presenta importantes problemas. Es difícil determinar qué cambió entre versiones, quién realizó una modificación y cuál es realmente la versión correcta.

Los sistemas de control de versiones permitieron resolver estos inconvenientes mediante el almacenamiento estructurado del historial de modificaciones.

La evolución puede resumirse en cuatro etapas:



- **Control de versiones local:** La información de las versiones se almacena en el equipo del desarrollador. El modelo local utiliza una copia de la base de control de versiones y una copia de los archivos del proyecto. Este tipo es el más sencillo y no es recomendable cuando se trabaja en equipo ya que todos tienen que acceder a los mismos archivos.
- **Control de versiones centralizado:** Un servidor central contiene el repositorio principal y los desarrolladores acceden a él.
- **Control de versiones distribuido:** Cada desarrollador dispone de una copia completa del repositorio y de su historial.
- **Plataformas colaborativas:** Servicios como GitHub, GitLab y Bitbucket incorporan repositorios Git junto con herramientas de colaboración, revisión de código, gestión de proyectos y automatización.

Esta evolución refleja el paso desde una preocupación centrada exclusivamente en "guardar versiones" hacia una concepción integral de la gestión del ciclo de vida del software.

Repositorios Centralizados

Concepto

- Los sistemas de control de versiones centralizados utilizan un único servidor principal donde se almacena el repositorio y su historial. Los usuarios se conectan a ese servidor para obtener archivos y subir sus cambios.
-
- Los desarrolladores trabajan con copias de los archivos y se comunican con el servidor central para obtener o registrar modificaciones.
- Entre los ejemplos tradicionales pueden mencionarse CVS, Subversion (SVN) y Perforce. Git identifica estos sistemas como ejemplos de sistemas de control de versiones centralizados.
- La arquitectura puede representarse conceptualmente de la siguiente manera:
- Un solo punto de verdad y de fallo (si el servidor cae, nadie puede trabajar ni acceder al historial).
- Requiere conexión constante al servidor para la mayoría de las operaciones.
- Más simple de administrar y entender.

Características	Ventajas	Desventajas
• Existencia de un repositorio central. • Historial almacenado principalmente en el servidor. • Dependencia de la comunicación con el servidor. • Administración centralizada. • Control de permisos. • Acceso común al código. • Facilidad para establecer políticas centralizadas.	• Administración relativamente sencilla. • Existencia de una fuente central de información. • Control de permisos centralizado. • Facilidad para determinar cuál es el repositorio principal. • Adecuación para determinadas estructuras organizacionales.	La principal desventaja es la dependencia del servidor central. Si el servidor deja de funcionar, pueden verse afectadas las operaciones de colaboración y registro de cambios. Además, una falla grave en el almacenamiento central puede comprometer el historial completo si no existen mecanismos adecuados de respaldo. No está disponible localmente, por lo que siempre es necesario estar conectado a una red para realizar cualquier acción. Como todo está centralizado, si el servidor se rompe provocará la pérdida completa del proyecto.

Repositorio Distribuido

En un sistema distribuido, cada desarrollador posee una copia del repositorio que contiene el historial del proyecto. Por lo tanto, no se limita a descargar una copia de los archivos actuales, sino que obtiene la información necesaria para trabajar con el historial del proyecto.

Git es uno de los principales ejemplos de sistemas distribuidos. También existen otras tecnologías distribuidas, como Mercurial.

Características	Ventajas	Desventajas
• Cada desarrollador puede disponer de un repositorio local completo. • Los commits pueden realizarse localmente. • Es posible trabajar sin conexión permanente al servidor remoto. • El historial se replica. • Se pueden utilizar múltiples repositorios remotos. • Se facilita el trabajo mediante ramas. • Se favorece la colaboración distribuida.	• Mayor independencia del servidor. • Trabajo local. • Mayor velocidad para determinadas operaciones. • Posibilidad de realizar commits sin conexión. • Existencia de múltiples copias del historial. • Flexibilidad para trabajar con diferentes flujos de desarrollo. • Facilidades para equipos distribuidos geográficamente.	• Mayor complejidad conceptual. • Necesidad de comprender ramas y fusiones. • Posibilidad de conflictos al integrar cambios. • Requieren buenas prácticas para evitar historiales desorganizados. • La existencia de múltiples copias requiere políticas adecuadas de seguridad y acceso.

Repositorio centralizado vs. repositorio distribuido

Aspecto	Centralizado	Distribuido
Historial completo	Solo en el servidor	En cada copia local
Trabajo offline	Limitado	Total
Punto único de fallo	Sí	No
Velocidad de operaciones	Depende de la red	Mayoría son locales, muy rápidas
Ejemplo típico	SVN	Git

Características de un repositorio

Control de versiones:

guarda cada modificación como una versión identificable.

Historial de cambios (log):

Registro cronológico de todos los commits.

Trazabilidad:

Permite saber quién hizo cada cambio y por qué (autor, fecha, mensaje).

Ramificación (branching):

Crear líneas de desarrollo paralelas.

Fusión (merging):

Combinar cambios de distintas ramas.

Reversión (rollback):

Volver a un estado anterior del proyecto.

Colaboración:

Múltiples desarrolladores trabajando sobre el mismo proyecto de forma ordenada.

Backup/respaldo:

Al mantener copias (locales o remotas), actúa como resguardo del proyecto.

Gestión de conflictos:

Herramientas para resolver cambios contradictorios sobre un mismo archivo.

Clasificación de repositorios

Se pueden clasificar según distintos criterios:

Según la arquitectura:

- Centralizados: Existe un servidor central que concentra la información. (SVN, CVS)
- Distribuidos: Cada copia local puede contener el historial completo del proyecto. (Git, Mercurial)

Según el acceso:

- **Públicos:** accesibles por cualquier persona (open source). Su contenido puede ser consultado por usuarios externos, de acuerdo con las condiciones establecidas por la plataforma.
- **Privados:** acceso restringido a determinados usuarios o a un equipo u organización. GitHub, por ejemplo, permite establecer repositorios públicos o privados y administrar los permisos de acceso.

Según el alojamiento:

- **Locales:** alojados en la propia máquina o servidor interno del desarrollador.
- **Remotos / en la nube:** alojados en plataformas externas (GitHub, GitLab, Bitbucket).

Repositorios híbridos

- Combinan repositorios locales y remotos.
- Git utiliza precisamente este esquema de trabajo, donde el desarrollador puede disponer de un repositorio local y sincronizarlo con uno o varios repositorios remotos.

Según el contenido/propósito:

- Repositorios de código fuente.
- Repositorios de paquetes/dependencias (npm, Maven, PyPI).
- Repositorios de contenedores (Docker Hub).

Plataformas de repositorios

Una característica fundamental de Git es que utiliza un modelo distribuido. Al clonar un repositorio, el desarrollador obtiene una copia del historial necesario para trabajar con el proyecto.

Es importante diferenciar tres conceptos:

- Git: sistema de control de versiones.
- Repositorio Git: estructura que almacena la información versionada.
- GitHub/GitLab/Bitbucket: plataformas que permiten alojar repositorios Git y proporcionan funcionalidades adicionales.

Esta diferencia es fundamental para comprender correctamente el ecosistema.

GitHub

- Un repositorio de GitHub contiene el código, los archivos y el historial de revisiones del proyecto. También permite discutir y administrar el trabajo asociado al repositorio.
-
- Propiedad de **Microsoft**.
- La plataforma más popular para proyectos open source.
- Funciones destacadas:
 - Pull Requests,
 - GitHub Actions (CI/CD),

-
- GitHub Pages,
 - Issues,
 - Projects (kanban),
 - Copilot integrado,
 - Branches. Discussions.
 - Projects.
 - Revisión de código.
 - Protección de ramas.
 - Gestión de permisos.
 - Commits.
 - Fuerte enfoque en comunidad y colaboración pública.
 - Las Pull Requests permiten proponer, discutir, revisar y fusionar cambios. Además, pueden integrarse con verificaciones automatizadas que permiten comprobar el estado de pruebas y procesos de integración antes de realizar un merge.

GitHub Actions

GitHub Actions permite automatizar workflows de desarrollo directamente desde el repositorio. Puede utilizarse para implementar procesos de integración continua y entrega continua, incluyendo compilación, pruebas y despliegue.

Por lo tanto, GitHub puede ser utilizado no solamente como repositorio de código sino como parte de una infraestructura integral de desarrollo de software.

GitLab

- GitLab es una plataforma orientada a la gestión integral del ciclo de vida del desarrollo de software. En GitLab, el trabajo se organiza principalmente mediante proyectos, dentro de los cuales se almacenan archivos, código, ramas, commits, tags, documentación, configuraciones y otros elementos relacionados.
- GitLab permite crear proyectos nuevos, importar proyectos existentes o inicializar repositorios Git. También proporciona diferentes opciones de visibilidad y funcionalidades relacionadas con seguridad.
- Puede usarse en la nube (gitlab.com) o **autoalojado (self-hosted)** en servidores propios, lo cual es una gran ventaja para empresas que necesitan control total.
- Una característica importante de GitLab es su orientación hacia **DevSecOps**, integrando desarrollo, operaciones y seguridad en el ciclo de vida del software.
- Integra un ciclo DevOps completo: repositorio, CI/CD, gestión de proyectos, seguridad, todo en una misma plataforma.
- Muy usado en entornos corporativos por su capacidad on-premise.
- Puede utilizarse como servicio en la nube o mediante instalaciones administradas por las propias organizaciones, según la modalidad contratada.
- Entre sus funcionalidades se encuentran:
 - Repositorios Git.
 - Branches.
 - Commits.
 - Merge Requests.
 - Issues.
 - Gestión de proyectos.

-
- CI/CD.
 - Gestión de seguridad.
 - Integración con herramientas externas.
 - GitLab Pages.
 - Gestión de releases.
 - Opciones de despliegue.

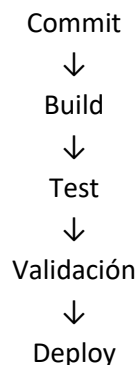
Bitbucket

- Bitbucket es una plataforma de Atlassian orientada principalmente al alojamiento y colaboración sobre repositorios Git.
- La documentación de Bitbucket Cloud contempla funciones para crear, importar y clonar repositorios, así como trabajar con branches, commits y Pull Requests.
- Propiedad de **Atlassian**.
- Se integra fuertemente con otras herramientas de Atlassian como **Jira** y **Trello**.
- Ofrece repositorios Git (y anteriormente soportaba Mercurial).
- Buena opción para equipos que ya usan el ecosistema Atlassian.
- Una característica especialmente relevante es su integración con **Jira**, que permite vincular actividades de desarrollo con tareas y procesos de gestión de proyectos.
- Entre sus características principales se encuentran:
 - Repositorios Git.
 - Branches.
 - Commits.
 - Pull Requests.
 - Revisión de código.
 - Gestión de incidencias.
 - Integración con herramientas del ecosistema Atlassian.
 - Bitbucket Pipelines.
 - Control de acceso.

Bitbucket Pipelines

Bitbucket Pipelines es un servicio integrado de CI/CD que permite automatizar procesos de construcción, pruebas y despliegue. La configuración de los pipelines se define mediante un archivo YAML ubicado en el repositorio.

Esto permite implementar flujos como:



Además, Bitbucket permite definir pipelines asociados a ramas y Pull Requests, posibilitando automatizar diferentes procesos según el evento que ocurra en el repositorio.

Resumen

Plataforma	Empresa	Fortaleza principal
GitHub	Microsoft	Comunidad open source, Actions
GitLab	GitLab Inc.	DevOps integrado, self-hosted
Bitbucket	Atlassian	Integración con Jira/Trello

Las tres plataformas utilizan Git como sistema de control de versiones, pero presentan diferencias relacionadas con su ecosistema, herramientas de integración y orientación.

Característica	GitHub	GitLab	Bitbucket
Control de versiones	Git	Git	Git
Repositorios públicos	Sí	Sí	Sí
Repositorios privados	Sí	Sí	Sí
Gestión de ramas	Sí	Sí	Sí
Revisión de código	Pull Request	Merge Request	Pull Request
Gestión de incidencias	Sí	Sí	Sí
CI/CD	GitHub Actions	GitLab CI/CD	Bitbucket Pipelines
Automatización	Alta	Alta	Alta
Gestión de proyectos	Sí	Sí	Sí
Integración Atlassian	Disponible	Disponible	Muy integrada
Seguridad	Amplias funcionalidades	Fuerte orientación DevSecOps	Integrada con ecosistema Atlassian
Código abierto	Ecosistema muy amplio	Amplio	Amplio
Opción autogestionada	GitHub Enterprise	GitLab Self-Managed	Bitbucket Data Center
Ecosistema destacado	Desarrollo colaborativo	DevOps/DevSecOps	Atlassian

No existe una plataforma universalmente superior. La elección depende de las necesidades de la organización, su infraestructura, herramientas existentes, requerimientos de seguridad y metodología de desarrollo.

Buenas prácticas para la utilización de repositorios

La utilización correcta de un repositorio requiere establecer normas y procedimientos.

Utilizar commits significativos

Cada commit debería representar un cambio concreto y comprensible.

Un mensaje como:

- cambios
- aporta poca información.
- Es preferible:
- Agregar validación de usuario en formulario de registro

Utilizar ramas

Las funcionalidades nuevas y correcciones deberían desarrollarse en ramas separadas cuando el flujo de trabajo del equipo así lo establezca.

Proteger la rama principal

La rama main o equivalente debería contar con mecanismos que eviten modificaciones directas no revisadas.

Utilizar Pull Requests o Merge Requests

Estos mecanismos permiten revisar cambios antes de integrarlos.

No almacenar información sensible

No deben incorporarse al repositorio:

- Contraseñas.
- Claves privadas.
- Tokens.
- Credenciales.
- Secretos de aplicaciones.
- Archivos con información confidencial.

Utilizar .gitignore

El archivo .gitignore permite evitar que determinados archivos sean incorporados al control de versiones.

Por ejemplo:

```
.env  
node_modules/  
*.log  
dist/
```

Documentar el proyecto

Un repositorio debería contener como mínimo un archivo README que explique:

- Objetivo del proyecto.
- Requisitos.
- Instalación.
- Configuración.
- Ejecución.
- Uso.
- Contribución.

Utilizar etiquetas de versión

Las versiones importantes pueden identificarse mediante tags:

v1.0.0

v1.1.0

v2.0.0

Automatizar pruebas

Siempre que sea posible, las pruebas deberían ejecutarse automáticamente ante determinados cambios.

Revisar periódicamente permisos

Los permisos deben adecuarse a las responsabilidades reales de cada integrante del proyecto.

Importancia de los repositorios en la Ingeniería de Software

Los repositorios desempeñan un papel fundamental en la Ingeniería de Software moderna.

Facilitan el trabajo colaborativo

Los equipos pueden trabajar simultáneamente sobre diferentes funcionalidades y posteriormente integrar los cambios.

Permiten controlar la evolución

El historial permite conocer cómo evolucionó el sistema y recuperar estados anteriores cuando sea necesario.

Mejoran la trazabilidad

Los cambios pueden relacionarse con funcionalidades, errores, tareas o requerimientos.

Favorecen la calidad

Los mecanismos de Pull Requests, Merge Requests y revisión de código permiten detectar problemas antes de incorporar modificaciones a ramas importantes. GitHub, por ejemplo, permite realizar revisiones y exigir aprobaciones antes del merge cuando se configuran las políticas correspondientes.

Facilitan DevOps

La integración entre repositorios y herramientas de CI/CD permite automatizar compilaciones, pruebas y despliegues.

Favorecen la continuidad del proyecto

El historial almacenado en múltiples copias constituye un mecanismo adicional de recuperación ante determinados problemas.

Facilitan el trabajo remoto

Los equipos distribuidos geográficamente pueden colaborar sobre un mismo proyecto sin necesidad de encontrarse físicamente en una misma ubicación.

Conclusión

Los repositorios de software constituyen actualmente un componente fundamental de la Ingeniería de Software. Su función supera ampliamente el simple almacenamiento de archivos, ya que permiten administrar versiones, registrar cambios, colaborar entre desarrolladores, organizar ramas, revisar código y automatizar procesos.

La evolución desde los sistemas de control de versiones locales hacia los modelos centralizados y posteriormente distribuidos respondió a la necesidad de mejorar la colaboración, disponibilidad y flexibilidad del desarrollo de software.

Los repositorios centralizados presentan una arquitectura basada en un servidor principal, lo que simplifica determinadas tareas administrativas, pero genera dependencia respecto de dicho servidor. Los sistemas distribuidos, por el contrario, permiten que cada desarrollador disponga de una copia completa del repositorio y de su historial, proporcionando mayor autonomía y flexibilidad.

Git se ha consolidado como una de las principales tecnologías de control de versiones distribuido. Su utilización se encuentra estrechamente relacionada con plataformas como GitHub, GitLab y Bitbucket. Estas plataformas agregan a Git diferentes servicios orientados a la colaboración y a la gestión integral del desarrollo.

GitHub se destaca por su amplio ecosistema de colaboración y automatización mediante GitHub Actions. GitLab presenta una fuerte orientación hacia la integración del ciclo de vida del desarrollo y las prácticas DevSecOps. Bitbucket, por su parte, posee una integración especialmente relevante con el ecosistema de herramientas de Atlassian y proporciona Bitbucket Pipelines para automatizar procesos de CI/CD.

En consecuencia, la elección de una plataforma no debería realizarse únicamente considerando su capacidad para almacenar código. Es necesario analizar factores como el tamaño y características del equipo, las necesidades de colaboración, los mecanismos de seguridad, la infraestructura disponible, las herramientas utilizadas por la organización y los procesos de integración y despliegue.

Finalmente, el conocimiento de repositorios y sistemas de control de versiones constituye una competencia fundamental para los profesionales de Ingeniería en Sistemas. La utilización adecuada de estas herramientas permite mejorar la organización del trabajo, reducir riesgos, favorecer la calidad del software y establecer prácticas compatibles con metodologías ágiles, DevOps y procesos modernos de desarrollo.

Bibliografía:

<https://www.opentix.es/blog/sistema-de-control-de-versiones/>
<https://www.datacamp.com/es/blog/what-is-a-repository>